

Universidade Estadual de Campinas (Unicamp)

Institute of Computing

MO434 – Deep Learning

Prof. Dr. Alexandre Xavier Falcão

Graph Relational Knowledge Distillation: Generalizing Relational KD to Arbitrary-Order Relations

Authors

Gabriel de Alcantara Bomfim Silveira RA: 197244

Rodrigo Antonio Godoy de Almeida RA: 324073

Summary. We propose Graph Relational Knowledge Distillation (Graph-RKD), a generalization of Relational Knowledge Distillation (RKD) to relations of arbitrary order: each minibatch gives rise to several complete, undirected, weighted graphs — one per sampled node set, with the number of nodes set by the relational order N — each summarized by a permutation-invariant descriptor (sorted node profiles or the classical-MDS spectrum) and matched between teacher and student. We study the method as a design space along three axes: the normalization of the graph distance matrix (per-graph, minibatch-level, none, and a hybrid scheme), the descriptor, and the matching objective (regression and contrastive). The method is implemented for deep metric learning (triplet task loss) and evaluated on Stanford Cars-196 and CUB-200 with disjoint-class splits, distilling ResNet-18 and ConvNeXt-Tiny teachers into a 0.67M-parameter student. The central comparison is Graph-RKD at orders $N \geq 3$ against the distance- and angle-wise RKD baselines (RKD-D, RKD-A), all added to the triplet task loss under a shared schedule and budget, with the relational weight λ_g tuned per configuration. We prove that at $N = 2$ the graph term either degenerates (under per-graph normalization) or reduces exactly to RKD-D (under minibatch normalization), so the pairwise case is represented by the RKD-D baseline and Graph-RKD is evaluated only at the orders where it is a distinct method. Deriving a rule that predicts the quality-optimal N from the minibatch size K is left as future work.

Abstract

Relational Knowledge Distillation (RKD) transfers the geometry of a teacher’s embedding space by matching pairwise distances and triplet-wise angles, but its tuple-based formulation is confined to relations of order two and three: the number of ordered N -tuples grows super-exponentially with N . We propose Graph Relational Knowledge Distillation (Graph-RKD), which generalizes relational transfer to arbitrary order by modeling each minibatch as several complete, undirected, weighted graphs — one per sampled node set, whose size is the relational order N — whose nodes are examples and whose edge weights are embedding distances. Because such graphs are determined by their node *sets*, ordering carries no information, reducing the relation space from a falling factorial to a binomial coefficient; permutation-invariant descriptors (sorted node profiles or the classical-MDS spectrum) and adaptive graph sampling keep the loss tractable. We treat Graph-RKD as a design space and study three axes: the normalization of the graph distance matrix (per-graph, minibatch-level, none, and a hybrid scheme), the choice of descriptor (profile, MDS), and the matching objective (regression, contrastive). On fine-grained retrieval (Stanford Cars-196 and CUB-200, disjoint-class splits), distilling ResNet-18 and ConvNeXt-Tiny teachers into a 0.67M-parameter ConvNeXt-inspired student, we compare Graph-RKD at orders $N \geq 3$ against the distance- and angle-wise RKD baselines (RKD-D, RKD-A), all added to a triplet task loss under a shared schedule and budget, with the relational weight tuned per configuration. We show analytically that the pairwise case $N = 2$ either degenerates under per-graph normalization or reduces exactly to RKD-D under minibatch normalization, so RKD-D represents that order and Graph-RKD is evaluated only where it is a distinct method ($N \geq 3$). A methodological finding recurs throughout: a multi-term distillation loss can zero out *silently*, detectable only by per-term logging. Deriving a rule that predicts the quality-optimal N from the minibatch size K is outlined as future work.

1 Introduction

Modern image classifiers such as VGG [1], ResNet [2], and ConvNeXt [3] share a common macrostructure: an encoder that produces a spatial feature map, a global average pooling (GAP) operation that collapses the spatial dimensions, and a linear classifier. While highly accurate, these backbones are computationally heavy in both parameter count and inference cost, which limits their deployment on resource-constrained devices and motivates the search for compact models that retain the accuracy of their larger counterparts.

Knowledge distillation addresses this need by training a small student network to imitate a larger pretrained teacher. The paradigm has evolved through several notions of *what* should be transferred. The seminal formulation of Hinton et al. [4] transfers knowledge at the output level, matching the teacher’s temperature-softened logits. FitNets [5] pushes supervision inward, regressing the teacher’s intermediate feature maps, and Attention Transfer [6] aligns the spatial attention maps derived from those activations. All three align *individual* representations, example by example.

Relational Knowledge Distillation (RKD) [7] marked a conceptual shift: rather than matching the

representation of each example in isolation, it transfers the *mutual relations* between examples — pairwise distances and triplet-wise angles in the embedding space. This relational view is powerful, but the original formulation is confined to relations of order two and three. Generalizing it to higher-order relations of size N is combinatorially prohibitive: the number of ordered N -tuples drawn from a minibatch grows super-exponentially with N , and much of that count is redundant because the relational structure of a set of examples does not depend on the order in which they are listed.

This work proposes *Graph Relational Knowledge Distillation* (Graph-RKD), a generalization of RKD to relations of arbitrary order. Our central idea is to model each minibatch as a set of complete, undirected, weighted graphs — one per sampled node set, each with N nodes, where the relational order N is a hyperparameter — in which every node is an example and every edge weight is the distance between the embeddings of its endpoints. Because such a graph is fully determined by its *set* of nodes, the ordering of nodes and edges carries no information, which removes the redundancy of the tuple formulation and reduces the space of relations from a falling factorial to a binomial coefficient. To keep the loss tractable even when this reduced space remains large, we summarize each graph with a permutation-invariant embedding and train the student over a small set of graphs sampled afresh at every iteration. Following the project’s suggestion to compare and adapt to the scheme of Park et al. [7], we instantiate the method in RKD’s native deep-metric-learning setting and evaluate it on two fine-grained retrieval benchmarks, Stanford Cars-196 [8] and CUB-200 [9], against the distance- and angle-wise RKD variants as baselines.

The remainder of this paper is organized as follows. Section 2 reviews the relevant literature on knowledge distillation, contrastive learning, and graph representation. Section 3 formalizes the distillation algorithms used as baselines. Section 4 presents the Graph-RKD method, its hypotheses, and its components. Section 5 describes the datasets, Section 6 the experimental setup, and Section 7 the results. Section 8 discusses the findings, Section 9 reports the difficulties encountered, and Section 10 concludes.

2 Related Work

2.1 Backbone Architectures

The teacher and student networks in this work are built from two families of convolutional backbones. ResNet [2] introduced residual connections that enable the stable training of very deep networks and remains a strong, widely used baseline; we adopt its lightweight ResNet-18 variant as one teacher. ConvNeXt [3] modernizes the convolutional design with larger kernels, inverted bottlenecks, and transformer-inspired training recipes, matching or surpassing vision transformers while retaining a purely convolutional structure; we use its ConvNeXt-Tiny variant as the second teacher and a compact ConvNeXt-style network as the student. Both teacher families share the canonical encoder–GAP–classifier macrostructure, producing a pooled embedding that our method treats as the node representation of each example.

Teacher and student embed the same sampled node sets; descriptors are matched.

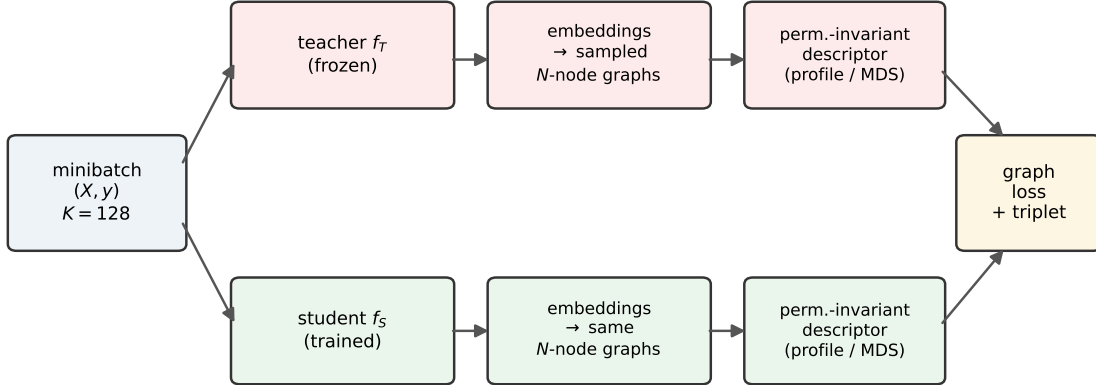


Figure 1: Overview of Graph-RKD. Each minibatch is embedded by the frozen teacher and the trained student; the *same* sampled N -node node sets are turned into complete weighted graphs on both sides, summarized by a permutation-invariant descriptor (profile or MDS), and matched by the graph loss (regression or contrastive) added to the triplet task loss. The descriptor dimensionality depends only on the order N , so teacher and student spaces of different widths need no projection layer.

2.2 Knowledge Distillation

The field of knowledge distillation was established by Hinton et al. [4], who proposed transferring the predictive behavior of a larger teacher network into a smaller student through softened output distributions, exposing the relative beliefs across classes encoded in the temperature-softened logits. Building on this output-level formulation, Romero et al. [5] proposed FitNets, which push supervision earlier in the network by regressing the teacher’s intermediate feature maps through learned “hint” layers, while Zagoruyko and Komodakis [6] refined this idea by transferring the compact spatial attention maps obtained by collapsing the channel dimension of those activations. These methods all align *individual* representations, example by example.

Park et al. [7] observed that the preceding approaches share this single template and proposed Relational Knowledge Distillation (RKD), which instead transfers the *mutual relations* between examples — pairwise distances and triplet-wise angles in the embedding space — and subsumes the individual paradigm as its unary special case. RKD is the conceptual foundation of the present work, and its distance-wise (RKD-D) and angle-wise (RKD-A) losses serve as our experimental baselines. Logit-based distillation [4] is discussed only as the output-level origin of the distillation progression, not as a baseline: our metric-learning setting exposes no shared classifier logits to transfer (Section 6.3). RKD’s formulation, however, is confined to relations of order two and three. Extending it to higher-order relations of size N is combinatorially prohibitive: the relations are defined over *ordered* tuples, so the number of distinct tuples grows super-exponentially with N , even though the relational structure of a set of examples is independent of the order in which they

are listed. This scaling barrier is the gap our method addresses.

2.3 Contrastive and Representation Learning

Contrastive learning maximizes agreement between related views of the data while separating unrelated ones, a principle formalized by the InfoNCE objective and its mutual-information bound [10] and scaled up by SimCLR [11] and MoCo [12]. The same idea was carried into distillation by Contrastive Representation Distillation (CRD) [13], which pulls the student’s representation toward the teacher’s for the same input and pushes it away from others. These objectives trace back to noise-contrastive estimation [14] and the negative-sampling scheme popularized by word embeddings [15]. One of our two distillation objectives adopts this contrastive view, but applies it at the level of *graph* embeddings rather than individual representations.

2.4 Graph Learning and Permutation Invariance

Training over large relational structures by sampling is standard practice in graph machine learning: GraphSAGE samples fixed-size neighborhoods to learn inductively on large graphs [16], and PinSage scales this strategy to web-scale recommendation [17]. A second requirement, that a set-level representation be invariant to the ordering of its inputs, is the defining property of set functions characterized by Deep Sets [18]. Our graph descriptors follow both principles: we sample graphs from each minibatch to keep training tractable, and we summarize each graph with permutation-invariant embeddings — one based on sorted node profiles and one on the eigenvalues of the classical multidimensional-scaling Gram matrix [19, 20], which are invariant under permutation similarity by construction.

3 Baseline Distillation Algorithms

This section formalizes the distillation methods against which Graph-RKD is compared. The distance- and angle-wise variants of Relational KD [7] (RKD-D, RKD-A) are our experimental baselines and are presented with their loss formulations. Logit-based distillation [4] and the feature-based methods that preceded RKD are reviewed only as context: the metric-learning setting adopted here exposes no shared classifier logits to distill, so logit matching is not applicable as a baseline.

3.1 Logit-based Distillation (Hinton) — context

The foundational formulation of knowledge distillation was introduced by Hinton et al. [4], who proposed transferring the generalization behavior of a larger teacher network to a smaller student by matching their output distributions. The key insight is that the soft probabilities produced by the teacher encode rich similarity structure across classes — often called *dark knowledge* — which is lost when training only with hard labels; both networks apply a temperature-softened softmax, and the student minimizes a weighted sum of the cross-entropy with the teacher’s softened outputs and a cross-entropy with the true labels. We include this method only as the output-level origin of the

distillation progression: it requires shared classifier logits, which the metric-learning setting of this work does not provide (Section 6.3), so it is not used as a baseline and we omit its loss formulation, in parallel with the feature-based methods below.

3.2 Other Feature-based Methods

Between logit matching and relational distillation, two feature-based methods bridged the gap and are noted here for context, though they are not used as baselines in our experiments. FitNets [5] was the first to distill knowledge from the teacher’s *intermediate* feature maps rather than its output, using a “hint” layer in the teacher to supervise a “guided” layer in the student through a regression objective, with a convolutional regressor reconciling the differing widths. Attention Transfer [6] refined this by transferring not the full feature map but a compact spatial attention map obtained by collapsing the channel dimension, a more transferable signal that was the first such method to scale to ImageNet. Both push supervision into the interior of the network while still aligning individual representations, the paradigm that Relational KD would subsequently generalize.

3.3 Relational Knowledge Distillation

Park et al. [7] depart from the individual-output paradigm shared by the preceding methods and propose Relational Knowledge Distillation (RKD), in which the unit of transferred knowledge is the structure of the embedding space rather than any single example’s representation. Their general formulation (Eq. 1) computes a relational potential ψ over N -tuples of examples in both networks and penalizes the discrepancy between teacher and student potentials, recovering conventional KD as the unary special case. Two instances are proposed: a distance-wise loss (RKD-D, Eqs. 2–3) that matches pairwise Euclidean distances normalized by the mean intra-batch distance, and an angle-wise loss (RKD-A, Eqs. 4–5) that matches the cosines of angles formed by embedding triplets, a higher-order and scale-invariant property that the authors found to converge faster and perform better. Because relations are invariant to the dimensionality of individual outputs, RKD requires no projection layer between teacher and student spaces, in contrast to FitNets [5]. The authors emphasize, however, that relational potentials are blind to absolute output values, so RKD cannot serve as the sole supervision where individual outputs are the quantity of interest; in *classification* they combine it with cross-entropy and find it most effective jointly with logit-based distillation [4] — a combination specific to the classification setting. In our metric-learning setting there are no shared logits, so RKD-D and RKD-A are instead added to a triplet task loss (Section 4.7). We note that RKD-A is intrinsically an *order-three* relation (an angle over an example triplet); Section 7.2 compares it directly against Graph-RKD at the matched order $N = 3$, isolating the difference between an angle-based and a distance-based description of the same three-node structure.

$$\mathcal{L}_{\text{RKD}} = \sum_{(x_1, \dots, x_n) \in \mathcal{X}^N} l(\psi(t_1, \dots, t_n), \psi(s_1, \dots, s_n)) \quad (1)$$

$$\psi_D(t_i, t_j) = \frac{1}{\mu} \|t_i - t_j\|_2, \quad \mu = \frac{1}{|\mathcal{X}^2|} \sum_{(x_i, x_j) \in \mathcal{X}^2} \|t_i - t_j\|_2 \quad (2)$$

$$\mathcal{L}_{\text{RKD-D}} = \sum_{(x_i, x_j) \in \mathcal{X}^2} l_\delta(\psi_D(t_i, t_j), \psi_D(s_i, s_j)) \quad (3)$$

$$\psi_A(t_i, t_j, t_k) = \cos \angle t_i t_j t_k = \langle \mathbf{e}^{ij}, \mathbf{e}^{kj} \rangle, \quad \mathbf{e}^{ij} = \frac{t_i - t_j}{\|t_i - t_j\|_2}, \quad \mathbf{e}^{kj} = \frac{t_k - t_j}{\|t_k - t_j\|_2} \quad (4)$$

$$\mathcal{L}_{\text{RKD-A}} = \sum_{(x_i, x_j, x_k) \in \mathcal{X}^3} l_\delta(\psi_A(t_i, t_j, t_k), \psi_A(s_i, s_j, s_k)) \quad (5)$$

Here $t_i = f_T(x_i)$ and $s_i = f_S(x_i)$ denote the teacher and student representations of example x_i , and $\mathcal{X}^N$ is the set of N -tuples of distinct training examples drawn from the mini-batch. The relational potential function $\psi(\cdot)$ measures a structural property of each tuple, and l_δ denotes the Huber loss, quadratic for residuals below one and linear beyond, which provides robustness to outlier relations. The normalization factor μ in Eq. 2 rescales distances by their mini-batch mean, making the loss focus on relative structure and accommodating scale differences between embedding spaces of distinct dimensionality. Conventional knowledge distillation is recovered from Eq. 1 as the special case $N=1$ with ψ the identity, so RKD is a strict generalization of the individual paradigm. In practice the relational losses are added to a task-specific objective as $\mathcal{L}_{\text{task}} + \lambda_{\text{KD}} \mathcal{L}_{\text{KD}}$, with one balancing factor per distillation term.

4 Methodology

This section presents Graph Relational Knowledge Distillation (Graph-RKD). We first describe the overall pipeline and the teacher training stage, then motivate the method by exposing the scaling limitation of relational distillation, introduce our graph-based reformulation, and detail each of its components: graph sampling, the permutation-invariant embeddings, the distillation objectives, and the selection of the relational order.

4.1 Overview

Graph-RKD follows the standard teacher–student distillation setting. A teacher embedding network is first fine-tuned on the target dataset and then frozen (Section 4.2). A compact student is then trained from scratch under a combination of a triplet task loss and a *relational graph loss* that transfers the geometric structure of the teacher’s embedding space (Sections 4.3–4.8). At each training step the minibatch is modeled as a set of complete weighted graphs over its examples; both networks embed the same graphs, and the student is trained to reproduce the teacher’s graph embeddings. At inference time the teacher is discarded: the trained student produces L_2 -normalized embeddings that are used directly for retrieval, with no dependence on the teacher.

4.2 Teacher Training

The teacher is an ImageNet-pretrained backbone — ResNet-18 or ConvNeXt-Tiny — fine-tuned as an *embedding network* rather than a classifier. Its output is an L_2 -normalized embedding, trained with a triplet loss (distance-weighted sampling, margin 0.2) on the disjoint training classes, using class-balanced (NPairs) batches, and evaluated by Recall@K. Once fine-tuning converges, the teacher is frozen: its parameters are fixed for the entire student distillation stage, and it is used only to produce reference embeddings. This preserves the pretrained knowledge while adapting the representation to the metric-learning objective, so the teacher provides the geometric structure that the student learns to reproduce.

4.3 From Relational Tuples to Relational Graphs

Relational Knowledge Distillation transfers the mutual relations between examples rather than their individual representations. In its original form these relations are defined over *ordered tuples*: pairwise distances over ordered pairs and angular relations over ordered triples. This formulation has two limitations. First, it fixes the relational arity to the small constants two and three. Second, generalizing it to relations of order N is combinatorially prohibitive, because the number of ordered N -tuples drawn without repetition from a minibatch of size K is the falling factorial

$$P(K, N) = \frac{K!}{(K - N)!} = K(K - 1) \cdots (K - N + 1), \quad (6)$$

which grows super-exponentially in the relational order N . Crucially, much of this count is redundant: the relational structure induced by a set of examples does not depend on the order in which they are listed.

4.4 Modeling the Structure as Graphs

We instead model the relational structure of a minibatch as a *complete, undirected, weighted graph*, in which each node is an example and each edge weight is the dissimilarity (distance) between the embeddings of its two endpoints. Because the graph is complete and undirected, it is fully determined by the *set* of its nodes, and the ordering of nodes and edges carries no information. The number of distinct N -node graphs that can be formed from a minibatch of size K is therefore the binomial coefficient

$$\binom{K}{N} = \frac{P(K, N)}{N!}, \quad (7)$$

i.e., a single row of Pascal’s triangle. Adopting the set view rather than the tuple view thus removes a factor of $N!$ — exactly the orderings that the tuple formulation counts redundantly. This is the central hypothesis of our method: that the order-invariant graph is the right object for higher-order relational transfer, both because it discards redundancy and because it admits the permutation-invariant descriptors introduced below.

The scale of this redundancy is stark. Figures 2 and 3 plot, on a linear axis, the number of ordered

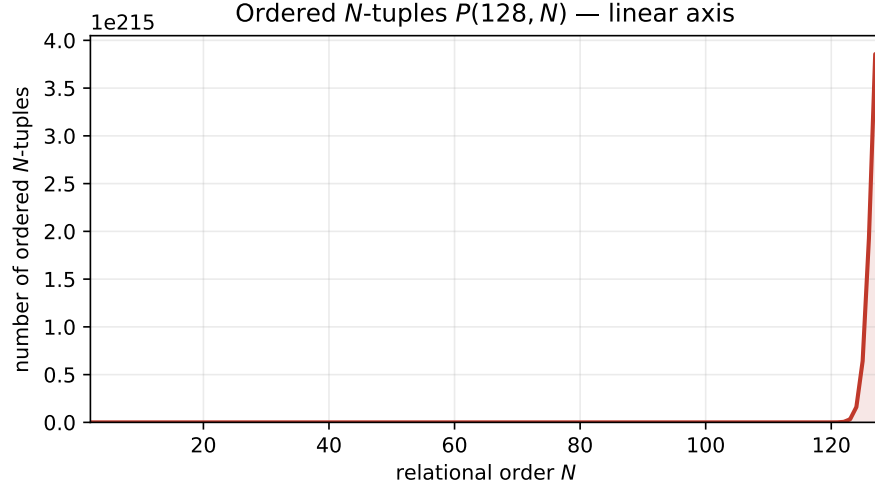


Figure 2: Number of ordered N -tuples $P(128, N)$ in a minibatch of $K = 128$, linear axis. The count is visually indistinguishable from zero across the range of practical orders and rises only in a near-vertical wall near $N = 128$, illustrating why a linear axis is uninformative for this quantity.

N -tuples $P(128, N)$ and the number of unique N -node graphs $\binom{128}{N}$ for a minibatch of $K = 128$. On a linear axis both are illegible over the range of interest: $P(128, N)$ is visually zero until it rises in a near-vertical wall (it reaches $\approx 3.9 \times 10^{215}$ at $N = 128$), and $\binom{128}{N}$ appears as a single narrow spike peaking at $N = 64$ ($\approx 2.4 \times 10^{37}$). Only on a logarithmic axis (Figure 4) are both curves readable simultaneously: the tuple count climbs monotonically while the graph count follows a symmetric, single-peaked profile, and the vertical distance between them — the factor $N!$ removed by the set view — widens with N . We stress that neither space is enumerable; the set view removes *redundancy* (and yields permutation-invariant descriptors), not enumeration cost, since both counts are astronomically large.

4.5 Tractability through Graph Sampling

The number of distinct graphs is nonetheless very large: $\binom{K}{N}$ is maximized at $N = K/2$ and reaches astronomical magnitudes (for $K = 128$, $\binom{128}{64} \approx 2.4 \times 10^{37}$), so enumerating all graphs to evaluate the loss is infeasible. We make the per-minibatch loss computable by *sampling* a small set of graphs at each step rather than enumerating them. Each step either partitions the shuffled minibatch into $\lfloor K/N \rfloor$ node-disjoint graphs or draws a set of random N -node subsets, and the sampled set is re-randomized every iteration. Under the random-subset scheme the gradient computed over the sample is an unbiased estimator of the gradient over the full graph space; the partition scheme trades some of this unbiasedness (it excludes overlapping node sets) for guaranteed single-pass coverage of every example. Either way the per-step cost is $O(G \cdot N^2)$ (with G the number of sampled graphs), independent of $\binom{K}{N}$.

Across a row of Pascal’s triangle, $\binom{K}{N}$ is single-peaked and, for large K , approximately Gaussian in N (de Moivre–Laplace [?]), centered at $N = K/2$ with standard deviation $\sqrt{K}/2$; equivalently $\log \binom{K}{N}$ is approximately parabolic in N near the peak. This describes the shape of the graph space

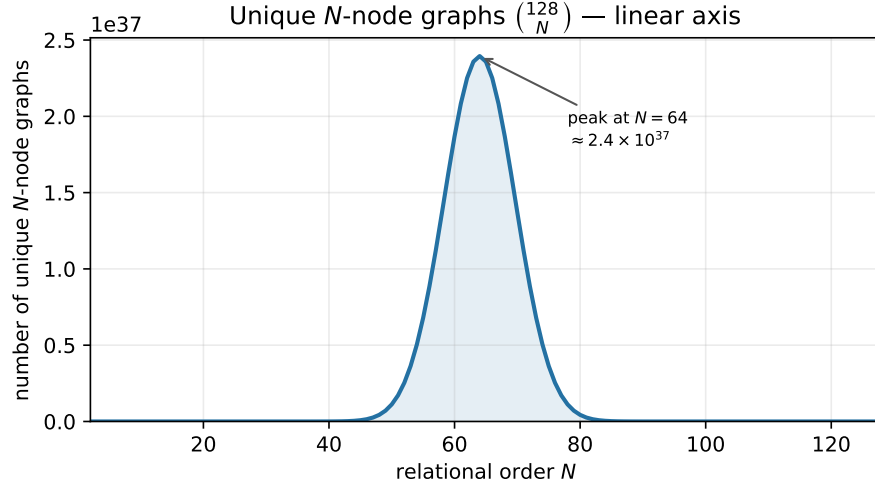


Figure 3: Number of unique N -node graphs $\binom{128}{N}$ in a minibatch of $K = 128$, linear axis. The count is a single symmetric spike peaking at $N = K/2 = 64$ ($\approx 2.4 \times 10^{37}$) and returning to 1 at $N = 128$; on a linear axis the small- N region where the method operates is crushed to the baseline.

and locates its maximum, though the approximation is quantitatively loose in the small- N tail ($N \ll K/2$) where our experiments operate, so we use it only as a shape guide, not a quantitative model. Because the graph space varies strongly with the relational order, we let the number of sampled graphs grow with that size rather than holding it fixed:

$$G(N) = \text{clamp}(\alpha \log_2 \binom{K}{N}, g_{\min}, g_{\max}), \quad (8)$$

with $\alpha = 0.5$ and $g_{\min} = \lfloor K/N \rfloor$ (at least one full-batch partition, ensuring coverage no worse than node-disjoint sampling), and an optional cap $g_{\max}$. The two terms are complementary and peak at opposite ends of N : the partition floor $\lfloor K/N \rfloor$ decreases monotonically (dominating at small N , where it guarantees cheap full-batch coverage), while the log-term follows the Gaussian-like profile of $\binom{K}{N}$ (dominating near $N = K/2$, where the space is most diverse and a single partition of ≈ 2 graphs would sample it far too thinly). We stress that this is a heuristic for matching sample *diversity* to the space, not a variance-reduction argument: the variance of a Monte-Carlo mean does not depend on population size. The justification is instead that the per-graph loss is most *heterogeneous* near $N = K/2$ (graphs there differ most from one another), so its population variance is largest there, warranting more samples to control the estimator; $\log_2 \binom{K}{N}$ is simply a convenient bounded function that peaks at $K/2$, and α is a tuning knob. Algorithm 1 summarizes the procedure.

Algorithm 1 Adaptive graph sampling

Require: batch size K , order N , α , $g_{\min}$ (default $\lfloor K/N \rfloor$), $g_{\max}$

- 1: $G \leftarrow \text{round}(\alpha \cdot \log_2 \binom{K}{N})$
 - 2: $G \leftarrow \text{clamp}(G, g_{\min}, g_{\max})$
 - 3: **return** G random N -subsets of $\{1, \dots, K\}$ $\triangleright$ or $\lfloor K/N \rfloor$ disjoint, if partition
-

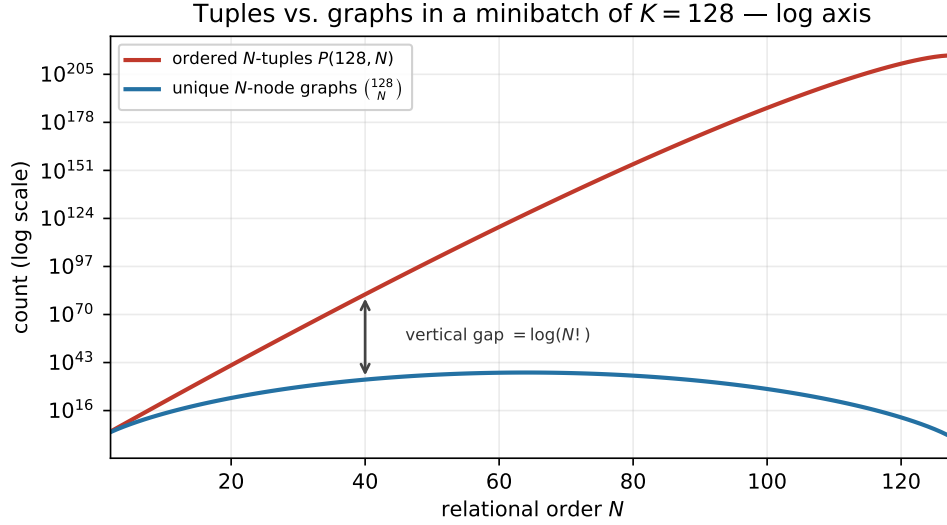


Figure 4: Ordered tuples versus unique graphs for $K = 128$ on a logarithmic axis. Both are legible here: $P(128, N)$ (red) is monotonic and saturating, while $\binom{128}{N}$ (blue) is single-peaked and approximately Gaussian in N (centered at $N = K/2$, cf. Section 4.5). The vertical gap between the curves equals $\log(N!)$ — the redundancy the set view removes.

4.6 Permutation-Invariant Graph Embeddings

Each sampled graph is summarized by a fixed-length descriptor computed from its dissimilarity matrix $D \in \mathbb{R}^{N \times N}$. The descriptor must be invariant to permutations of the node indexing (relabeling rows and columns of D leaves the graph unchanged) while remaining sensitive to its structure. We use two such embeddings.

The *ordered node-profile embedding* sorts the off-diagonal entries of each row into a per-node neighborhood profile, orders the N profiles lexicographically, and flattens the result into a vector of size $N(N - 1)$. Sorting within each row and then across nodes destroys the arbitrary node labeling, which is what makes the descriptor permutation-invariant; the price is that node correspondence (which edge joins which pair) is discarded, so two structurally different weighted graphs with the same multiset of sorted profiles collide. The *spectral (classical-MDS) embedding* applies classical multidimensional scaling: it double-centers the squared dissimilarity matrix into a Gram matrix $B = -\frac{1}{2}JD^{(2)}J$ with $J = I - \frac{1}{N}\mathbf{1}\mathbf{1}^\top$, recovering the inner-product matrix of the point configuration that best reproduces the distances; its eigenvalues (the squared extents along the principal axes), sorted in decreasing order, give a descriptor of size N that is invariant under permutation similarity by construction. Its price is the converse: the spectrum determines the graph only up to cospectral equivalence, so isospectral graphs collide. Both descriptors are therefore *lossy* summaries; a small worked example is instructive: for a three-node graph with normalized edges (a, b, c) the profile is the sorted-and-flattened list of each node’s two incident edges, whereas the MDS spectrum is the two nonzero eigenvalues of the double-centered 3×3 matrix. Table 1 makes this concrete, and also shows why $N = 2$ is excluded: under per-graph normalization the two-node descriptor is constant regardless of the edge.

Table 1: Worked descriptor examples for small graphs, under per-graph normalization (each matrix divided by its own mean off-diagonal entry). At $N = 2$ the single edge normalizes to 1, so *both* descriptors are constant for every pair — the degeneration that makes $N = 2$ uninformative (cf. Lemma 1). At $N = 3$ the descriptors vary with the (normalized) triangle, so the order carries genuine relational signal. Here $\bar{d}$ denotes the mean of the three edges and $\hat{a} = a/\bar{d}$, etc.

Order	Normalized edges	profile (dim $N(N-1)$)	mds spectrum (dim N)
$N = 2$	(1)	(1, 1)	(0.5, 0)
$N = 3$	$(\hat{a}, \hat{b}, \hat{c})$	sorted per-node incident-edge pairs	two nonzero eigenvalues of $-\frac{1}{2}JD^{(2)}J$

Profile versus MDS. The two descriptors trade off along several axes; at the feasible orders here ($N \leq 17$) neither is a compute concern (the largest profile is 272-dimensional). The *profile* retains every edge weight and is *operationally interpretable* — each component is a concrete normalized distance traceable to a specific example pair, which is precisely the property that let per-term inspection localize failures — but its dimension grows as $N(N-1)$, so its raw loss magnitude is larger and order-dependent, coupling it to the weight λ_g (Section 4.7); and its sort introduces non-smoothness at edge-weight ties. The *MDS* spectrum is compact (N -dimensional) and *theoretically interpretable* (principal-axis variances) but non-local, lossier (cospectral collisions), and numerically fragile where eigenvalues are near-degenerate. We report both and let the experiments adjudicate which transfers better in which regime (Section 7). Table 2 summarizes the trade-offs.

Table 2: Profile versus MDS descriptor, scoped to the feasible orders ($N \leq 17$). Dimensionality is not a compute concern in this regime; its only live consequence is the coupling to the loss weight λ_g . Fidelity and explainability move together: the profile keeps the raw edges (traceable and less lossy) while MDS compresses to a spectrum (non-local and subject to cospectral collisions).

Property	profile	mds
Dimension	$N(N-1)$ (e.g. 272 at $N=17$)	N
Information retained	all edges; loses node correspondence	spectrum only; cospectral collisions
Explainability	operational (per-edge, traceable)	theoretical (principal-axis variances)
$N = 2$ reduction (batch norm)	exact RKD-D (linear)	squared-distance variant
Numerical risk	kinks at edge-weight ties	instability at repeated eigenvalues
λ_g coupling	stronger (larger raw loss)	milder

Normalization of the distance matrix. Before computing a descriptor, the graph distance matrix may be rescaled. This normalization is a first-class design axis of the method, not an incidental step, and we study four schemes: (i) *per-graph* normalization, dividing each matrix by its own mean off-diagonal entry (making descriptors scale-invariant and directly comparable between teacher and student of different embedding widths); (ii) *minibatch-level* normalization, dividing every graph in a step by the single mean pairwise distance μ of the whole batch (Eq. 2), which preserves cross-graph scale and is the more stable, lower-variance denominator; (iii) *none* (raw distances), preserving absolute scale; and (iv) a *hybrid* scheme that uses the μ of (ii) as the denominator while keeping the descriptor itself scale-invariant (e.g. MDS-spectrum ratios or cross-graph z-scoring), aiming to recover both cross-graph scale consistency and teacher/student invariance. A key observation

motivates studying all four: at $N \geq 3$ the schemes are genuinely different objects, not related by a global constant, so the choice is not cosmetic. Per-graph normalization, moreover, *degenerates* at $N = 2$: a two-node graph has a single edge, so dividing by its own mean yields the constant matrix $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ and hence a constant descriptor for every pair. Under minibatch normalization, by contrast, the $N = 2$ descriptor recovers exactly the RKD-D potential; we make this precise in Lemma 1 and use it to justify omitting $N = 2$ from the study of Graph-RKD as a distinct method (Section 4.3). Algorithm 2 details both descriptor variants; the normalization scheme is a parameter of the `normalize` step.

Algorithm 2 Permutation-invariant graph embedding

Require: $D \in \mathbb{R}^{N \times N}$ (distances); method $\in \{\text{PROFILE}, \text{MDS}\}$; normalize

- 1: **if** normalize **then** ▷ scale-invariance (teacher vs. student)
- 2: $D \leftarrow D / \text{mean}_{i \neq j} D_{ij}$
- 3: **end if**
- 4: **if** method = PROFILE **then**
- 5: **for** each row i **do**
- 6: $p_i \leftarrow \text{sort_ascending}(\{D_{ij} : j \neq i\})$ ▷ neighborhood profile
- 7: **end for**
- 8: order $\leftarrow \text{lex_argsort}(p_1, \dots, p_N)$ ▷ canonical node order
- 9: **return** flatten($[p_{\text{order}(1)}; \dots; p_{\text{order}(N)}]$) ▷ in $\mathbb{R}^{N(N-1)}$
- 10: **else** ▷ MDS: classical MDS spectrum
- 11: $J \leftarrow I - \frac{1}{N} \mathbf{1}\mathbf{1}^\top$
- 12: $B \leftarrow -\frac{1}{2} J(D \odot D)J$ ▷ double-centered Gram
- 13: **return** eigenvalues(B) sorted descending ▷ in $\mathbb{R}^N$
- 14: **end if**

Lemma 1 (Graph-RKD reduces to RKD-D at $N = 2$). *Let each two-node graph on examples (i, j) carry the single edge weight $d_{ij} = \|z_i - z_j\|_2$, and let normalization be at the minibatch level, $\tilde{d}_{ij} = d_{ij}/\mu$ with μ the mean pairwise distance over the batch, so $\tilde{d}_{ij}$ equals the RKD-D potential ψ_D of Eq. 2. Then: (i) the profile descriptor is $g_{ij} = (\tilde{d}_{ij}, \tilde{d}_{ij})$, a linear bijection of $\tilde{d}_{ij}$, and the regression objective gives $\|g_{ij}^s - g_{ij}^t\|_p = c_p |\tilde{d}_{ij}^s - \tilde{d}_{ij}^t|$ with $c_1 = 2$, $c_2 = \sqrt{2}$, i.e. Graph-RKD coincides with distance-wise RKD up to a constant absorbed into λ_g ; (ii) the MDS descriptor is $g_{ij} = (\frac{1}{2}\tilde{d}_{ij}^2, 0)$, matching $\tilde{d}^2$ rather than $\tilde{d}$, a squared-distance variant of RKD-D.*

The equivalence holds per pair and in expectation under full-pair sampling; published RKD-D uses the Huber potential, so (i) agrees with it in the large-residual regime and differs only near zero. Under *per-graph* normalization the same $N = 2$ cell instead collapses to a constant descriptor. In every normalization scheme, then, $N = 2$ is either degenerate or equivalent to RKD-D — so it carries no information beyond the RKD-D baseline, and we evaluate Graph-RKD only at the orders $N \geq 3$ where it is a distinct, higher-order method (Section 4.3), with RKD-D serving as the representative of the pairwise case. A proof is a direct computation on the 2×2 matrix and is omitted for brevity.

4.7 Distillation Objectives

Given the same sampled node set on both sides, we transfer the teacher’s relational geometry to the student through two objectives, both evaluated in this work. The *regression* objective minimizes the Minkowski distance between the student’s and the teacher’s graph embeddings, $\|g_s - g_t\|_p$, directly matching the teacher’s geometry graph by graph; it is simple and parameter-free. The *contrastive* objective is an InfoNCE formulation in the spirit of contrastive representation distillation [13]: the student’s embedding of graph i is the anchor, the teacher’s embedding of the same graph i is the positive, and the teacher’s embeddings of other graphs $j \neq i$ provide the negatives, of which M are sampled per anchor. The loss is the cross-entropy of the resulting $(1+M)$ -way classification with the positive as the target, scaled by a temperature τ . This bounds the number of inter-graph comparisons to $G \cdot M$.

Because the descriptors are deterministic and parameter-free, it is worth stating why a contrastive objective is well-defined here at all. The descriptor is not learned; the contrastive loss optimizes the student *network* f_s through the descriptor, whose only requirement is differentiability (sorting is piecewise-linear; the eigendecomposition has well-defined derivatives away from degeneracies), so gradients flow back into f_s exactly as under the regression objective. The descriptor plays the role of a frozen, parameter-free projection head; determinism is orthogonal to gradient flow. We also temper the usual claim that contrastive transfer is inherently superior: that argument, imported from settings with no explicit target, is weaker here, since a regression target exists and the triplet loss already provides representation spreading. The setup-specific reason contrastive may nonetheless help is that InfoNCE is similarity-based and therefore scale-normalized, so it should be far less sensitive to the raw-loss magnitude that couples the regression objective to λ_g — which makes it most interesting precisely in the regime where a mis-scaled regression weight fails. Note that the contrastive objective inherits the same $N = 2$ degeneration under per-graph normalization (a constant descriptor makes anchor, positive, and all negatives coincide), so like regression it is meaningful only at $N \geq 3$ or under non-degenerate normalization.

In the metric-learning setting both graph objectives are added to a *triplet task loss* on the student, with a warm-up over the first epochs that ramps a multiplier from 0 to 1 so the relational term reaches its full weight λ_g only once the student’s early embeddings are informative; there is no logit-based or cross-entropy term. The complete distillation step is given in Algorithm 3.

Classic baselines replace $\lambda_g \cdot \mathcal{L}_{\text{rel}}$ by $\lambda \cdot \mathcal{L}_{\text{RKD-distance}}$ or $\lambda \cdot \mathcal{L}_{\text{RKD-angle}}$, under the same triplet loss and warm-up schedule.

4.8 Choosing the Relational Order N

The relational order N is the central hyperparameter of the method. Its selection has two distinct parts, which we treat differently.

Algorithm 3 Graph-RKD distillation step (metric learning)

Require: student f_s , teacher f_t , minibatch (X, y) with class-balanced sampling, order N , weight λ_g , objective $\in \{\text{REG}, \text{CON}\}$, p , M , τ , epoch e , warm-up fraction w

- 1: $Z_s \leftarrow \text{normalize}(f_s.\text{embedding}(X))$ $\triangleright L_2$ embeddings ($B \times d_s$)
- 2: $Z_t \leftarrow \text{normalize}(f_t.\text{embedding}(X))$ (no grad) $\triangleright L_2$ embeddings ($B \times d_t$)
- 3: $\{S_1, \dots, S_G\} \leftarrow \text{sample graphs (Algorithm 1)}$
- 4: **for** $g = 1 \dots G$ **do** $\triangleright$ same node set on both sides
- 5: $e_s[g] \leftarrow \text{GraphEmbedding}(\text{pdist}(Z_s[S_g]))$ $\triangleright$ Algorithm 2
- 6: $e_t[g] \leftarrow \text{GraphEmbedding}(\text{pdist}(Z_t[S_g]))$ (no grad)
- 7: **end for**
- 8: **if** objective = REG **then**
- 9: $\mathcal{L}_{\text{rel}} \leftarrow \text{mean}_g \|e_s[g] - e_t[g]\|_p$
- 10: **else** $\triangleright$ contrastive (InfoNCE, CRD-style)
- 11: **for** g **do**
- 12: logits $\leftarrow [\text{sim}(e_s[g], e_t[g]), \{\text{sim}(e_s[g], e_t[h]) : h \in \text{negs}(g, M)\}]/\tau$
- 13: **end for**
- 14: $\mathcal{L}_{\text{rel}} \leftarrow \text{mean}_g \text{CrossEntropy}(\text{logits}, \text{target} = 0)$ $\triangleright$ positive at index 0
- 15: **end if**
- 16: rel $\leftarrow \min(1, e/(w \cdot \text{total_epochs}))$ $\triangleright$ warm-up: balances rel vs. triplet
- 17: $\mathcal{L} \leftarrow \text{Triplet}(Z_s, y) + \text{rel} \cdot \lambda_g \cdot \mathcal{L}_{\text{rel}}$
- 18: backprop and update f_s

Compute-budget ceiling (binary search). The per-step cost under the partition scheme is $E = K(N - 1)/2$ edges, so the largest order that fits a budget E has the closed form

$$N_{\max} \leq \frac{2E}{K} + 1, \quad (9)$$

i.e., the feasible order is inversely proportional to the batch size. Feasibility is a *monotonic*, *deterministic* predicate — if an order N is infeasible, so is every larger order — so the ceiling $N_{\max}$ is found exactly by binary search in $O(\log K)$ evaluations and without any training.

Quality-driven selection (budget-bounded log-spaced sweep). We deliberately do *not* use binary search on the validation quality $q(N)$. Binary search makes an irreversible left/right cut at each midpoint and is valid only when the search signal is monotonic and noiseless; neither holds here, since $q(N)$ may rise and then fall (a larger N yields richer relations but fewer graphs per batch and a harder optimization), and each $q(N)$ is a single, stochastic training estimate. Instead we evaluate a set of log-spaced candidate orders

$$\mathcal{C} = \{n_{\min}, b n_{\min}, b^2 n_{\min}, \dots, N_{\max}\}, \quad b = 2, \quad (10)$$

which contains $\approx \log_b N_{\max}$ points — the same training budget a binary search would spend — but reveals the shape of the quality- N curve without assuming monotonicity. Following Lemma 1, we omit $N = 2$ (represented by the RKD-D baseline) and start the sweep at the smallest genuinely higher-order value; we additionally include $N = 3$ as an explicit candidate — the arity-matched

counterpart to RKD-A — even though pure base- b spacing would skip it. Each candidate is trained with a short schedule over R seeds, and its score is the mean best validation mAP@R. We then select N^* either by $\arg \max$ or, for parsimony, by the *one-standard-error rule*: the smallest N whose mean score is within one standard error of the best, preferring cheaper (smaller- N) configurations when the difference is not statistically significant. A final long run is trained at N^* . This costs $O(R \log_b N_{\max})$ trainings, is robust to non-monotonic and noisy quality curves, and exposes the curve for inspection. Beyond selecting a single N^* , we characterize each order along convergence speed, seed stability, peak quality, and per-step cost, and report the order that is best *by trade-off* rather than by peak metric alone. Algorithm 4 states the full procedure.

A rule for $N^*(K)$ as future work. A natural question is whether the quality-optimal order can be predicted directly from the minibatch size K , as a quality counterpart to the closed-form compute ceiling $N_{\max} \leq 2E/K + 1$. Establishing such a rule would require sweeping K across datasets, teachers, descriptors, objectives, and normalization schemes — a combinatorial program large enough to constitute a separate study — so we leave it as future work. The per-order characterization at fixed $K = 128$ produced here is the groundwork such a study would build on.

Algorithm 4 Selecting N (budget-bounded log-spaced sweep)

Require: batch K , edge budget E , $n_{\min}$, base b , seeds R , rule $\in \{\text{ARGMAX}, 1\text{SE}\}$

- 1: $N_{\max} \leftarrow$ largest N with $K(N - 1)/2 \leq E$ $\triangleright = \lfloor 2E/K + 1 \rfloor$, binary search, no training
- 2: $\mathcal{C} \leftarrow \{n_{\min}, b n_{\min}, b^2 n_{\min}, \dots, N_{\max}\}$ $\triangleright$ sorted, unique
- 3: **for** $N \in \mathcal{C}$ **do** $\triangleright$ seed-averaged validation quality
- 4: vals $\leftarrow [\text{best_val_mAP@R}(\text{distill}(N, \text{short}, \text{seed} = r)) : r = 1 \dots R]$
- 5: mean[N] $\leftarrow$ mean(vals); sem[N] $\leftarrow$ std(vals)/ $\sqrt{R}$
- 6: **end for**
- 7: **if** rule = ARGMAX **then**
- 8: $N^* \leftarrow \arg \max_N \text{mean}[N]$
- 9: **else** $\triangleright$ one-standard-error rule
- 10: best $\leftarrow \arg \max_N \text{mean}[N]$
- 11: $N^* \leftarrow$ smallest $N \in \mathcal{C}$ with $\text{mean}[N] \geq \text{mean}[\text{best}] - \text{sem}[\text{best}]$
- 12: **end if**
- 13: **return** N^* $\triangleright$ final long run trained at N^*

4.9 Student Architecture

The student is a compact network built on the design principles introduced by ConvNeXt [3] — larger kernels, inverted bottlenecks, and the ConvNeXt block structure — rather than an official ConvNeXt variant; we refer to it below as the student for brevity. It has stage dimensions (24, 48, 96, 192) and depths (1, 1, 3, 1), totaling approximately 0.67M parameters and 0.11 GFLOPs at 224×224 input (multiply-accumulate convention) — against 11.69M parameters / 1.81 GFLOPs for ResNet-18 and 28.59M / 4.46 GFLOPs for ConvNeXt-Tiny, i.e., a $17\text{--}43\times$ reduction in parameters and $16\text{--}41\times$ in compute relative to its teachers. It is trained from scratch as an embedding network with the combined triplet and graph-loss objective; no logit-based, attention, or original relational distillation

terms are used.

The student’s final L_2 -normalized embedding is 192-dimensional, against 512 for ResNet-18 and 768 for ConvNeXt-Tiny. Because the graph loss compares permutation-invariant descriptors whose dimensionality depends only on the relational order N , the same descriptor applies unchanged across all three widths: teacher and student embeddings inhabit the same descriptor space with *no* projection layer between them, in contrast to FitNets [5], whose learned regressor exists precisely to reconcile differing widths. A from-scratch student trained with the triplet loss alone, under identical hyperparameters, serves as an ablation *floor* — it measures whether *any* relational term helps over the task loss alone — rather than as a comparison target; the comparison of interest is Graph-RKD against the RKD-D and RKD-A baselines (Section 6.3).

5 Datasets

We evaluate on two standard fine-grained deep-metric-learning and retrieval benchmarks. Stanford Cars-196 [8] contains 16,185 images of 196 car models, and Caltech-UCSD Birds-200-2011 (CUB-200) [9] contains 11,788 images of 200 bird species.

We use the standard *metric-learning split*, in which the training and test classes are *disjoint*: for Cars-196 the first 98 classes are used for training and the remaining 98 for testing, and analogously the first and last 100 classes for CUB-200. This protocol evaluates whether a model has learned to *measure similarity* rather than to classify a fixed label set, since the test classes are never seen during training. Because the disjoint split provides no validation set, we hold out a *class-disjoint* subset of the training classes as validation: 20% of the training *classes* (selected once by a fixed random seed, with all images of a held-out class going entirely to validation), so that validation classes too are unseen during training.

5.1 Evaluation Metrics

Relevance is binary — two images match if they share a class — and we report a small, purposeful set of retrieval metrics. **Recall@K** ($K \in \{1, 2, 4, 8\}$) measures the conventional metric-learning hit rate, i.e., whether a same-class neighbor appears among the top K retrieved items, and is kept for comparability with the deep-metric-learning literature. **R-Precision** and **mAP@R** are order-sensitive over all relevant items: for each query they score the full ranking of its R same-class items rather than only whether one appears in the top K . Following the metric-learning reality check of Musgrave et al. [21], which shows that Recall@1 is saturated and can be misleading, **mAP@R is our primary metric** and the sole model-selection signal (both the checkpoint and the relational order N). Recall@K is retained *only* for comparability with the deep-metric-learning literature, where it is the standard, and is reported as a secondary metric — following Musgrave et al., who likewise report but do not rely on it. We omit precision@K, F1@K, nDCG, and MRR, which add little beyond Recall@K and mAP@R under binary relevance and are not standard in this setting.

Model selection — the checkpoint kept for final evaluation and the relational order N — relies only on the validation split; the test set is read once, with the validation-selected checkpoint, and all

three metrics (Recall@K, R-Precision, and mAP@R) are reported on the test split.

6 Experimental Setup

Table 3 summarizes the experimental design as a grid over the method’s axes, together with the fair-comparison protocol shared by all students.

Table 3: The Graph-RKD experimental design. The method is studied as a grid over four axes; the five students share the architecture, teacher, splits, budget, task loss, and warm-up schedule, differing only in the relational term. Classic baselines use the original RKD weights and recipe; only Graph-RKD’s λ_g is tuned (per configuration, on validation). $N = 2$ is omitted throughout (Lemma 1).

Axis / factor	Levels
Normalization	per-graph, minibatch-level (μ_{batch}), none, hybrid
Relational order N	3, 4, 8, 16, 17 (at $K=128$; $N=2$ and $N=1$ excluded)
Descriptor	profile ($\in \mathbb{R}^{N(N-1)}$), mds ($\in \mathbb{R}^N$)
Objective	regression (Minkowski $p=2$), contrastive (InfoNCE; M , τ)
Dataset $\times$ Teacher	$\{\text{Cars-196, CUB-200}\} \times \{\text{ResNet-18, ConvNeXt-Tiny}\}$
Students compared	(1) triplet-only [floor]; (2) +RKD-D; (3) +RKD-A; (4) +RKD-D+RKD-A [1:2]; (5) +Graph-RKD [swept]
Classic weights	RKD-D = 25, RKD-A = 50 (prescribed, Huber, μ -norm)
Graph-RKD weight	λ_g tuned per configuration on validation
Selection metric	validation mAP@R (Recall@K reported, not selected on)
Final runs	multi-seed (≥ 3), mean $\pm$ sem

6.1 Preprocessing and Data Augmentation

All images are processed as 224×224 RGB tensors. Training uses `RandomResizedCrop(224)` followed by a random horizontal flip; at evaluation (validation and test) we resize the shorter side to 256 and take a central 224 crop. In both cases pixels are scaled to $[0, 1]$ and normalized with ImageNet statistics (mean (0.485, 0.456, 0.406), std (0.229, 0.224, 0.225)), matching the ImageNet-pretrained teachers. We deliberately do *not* use mixup or cutmix in the distillation runs: mixing samples would destroy the per-example correspondence that the graph loss needs to pair student and teacher on identical node sets. Pairwise edge weights are Euclidean distances between embeddings; the graph distance matrix is then normalized according to the scheme under evaluation — per-graph, minibatch-level, none, or hybrid (Section 4.6) — rather than by a single fixed rule.

6.2 Teacher Fine-tuning

Teachers are ImageNet-1k pretrained backbones — ResNet-18 or ConvNeXt-Tiny — fine-tuned as *embedding networks* with a triplet loss (distance-weighted sampling, margin 0.2) on the disjoint training classes, using L_2 -normalized embeddings and class-balanced (NPairs) batches. Checkpoint selection uses validation mAP@R, consistent with the rest of the pipeline (Section 5.1); Recall@K is

additionally reported. Optimization uses AdamW (lr 10^{-4} , weight decay 10^{-5}), 60 epochs, batch size 128, a cosine schedule with a 3-epoch warm-up, and mixed precision.

6.3 Student Distillation

The student (Section 4.9) is trained from scratch as an embedding network with the triplet loss plus the graph loss only — there is no Hinton-style logit distillation, since metric learning has no shared classifier logits. The relational graph term is balanced against the triplet by a warm-up that ramps a multiplier from 0 to 1 over the first 10% of epochs, so the relational term reaches its full weight λ_g only once the student’s early embeddings are informative (the effective weight ramps to λ_g , not to 1). Optimization uses AdamW (lr 10^{-3} , weight decay 0.05), a cosine schedule with a 5-epoch warm-up, 120 epochs, batch size $K = 128$, NPairs class-balanced sampling, stochastic depth 0.1, and mixed precision.

We compare five students of the same architecture, differing only in the relational term added to the shared triplet loss and schedule: (1) triplet only, an ablation *floor*; (2) triplet + RKD-D; (3) triplet + RKD-A; (4) triplet + RKD-D+RKD-A (the combined classic reference); and (5) triplet + Graph-RKD, our method, swept over the normalization, descriptor, objective, and order axes. To represent the classic baselines at full strength, we follow the original RKD recipe [7]: the distance potential is μ -normalized (Eq. 2), the potentials use the Huber loss, and we adopt the paper’s prescribed weights — RKD-D at 25, RKD-A at 50, i.e. the original 1:2 distance-to-angle ratio, which the combined student (4) also uses. These are the weights validated by the baselines’ authors, so no baseline is disadvantaged by our tuning. Graph-RKD, by contrast, has no prescribed weight, so its λ_g is tuned per configuration on the *validation* split (never on test), and we report its λ_g -sensitivity so the chosen value is transparent rather than cherry-picked. All five students share the teacher, splits, budget, task loss, and warm-up schedule.

6.4 Graph-Loss Hyperparameters

Table 4 lists the default configuration of the graph loss. We evaluate both the *regression* and *contrastive* objectives of Section 4.7 across the same grid, and both descriptor variants (**profile**, **mds**) separately; the relational weight λ_g is tuned per configuration on validation rather than fixed. Following Lemma 1, the order sweep omits $N = 2$ (represented by the RKD-D baseline) and runs $N \in \{4, 8, 16, 17\}$ from the log-spaced schedule plus an explicit $N = 3$ point, the arity-matched comparison to RKD-A. Under the adaptive sampling rule at $K = 128$, the effective number of graphs per step is $G = \lfloor K/N \rfloor$ or the description-length value of Eq. 8, whichever is larger (e.g. $G = 42, 32, 20, 33, 35$ for $N = 3, 4, 8, 16, 17$).

7 Results

[This section is intentionally left blank. It is to be written from the experiments specified below. The boxed guides in each subsection define the runs, metrics, and analysis; they are instructions

Table 4: Default graph-loss hyperparameters.

Hyperparameter	Value
edge metric	Euclidean (distance matrix normalization swept: per-graph / minibatch / none / hybrid)
embedding	profile ($\in \mathbb{R}^{N(N-1)}$) or mds ($\in \mathbb{R}^N$)
objective	regression (Minkowski, $p = 2$) or contrastive (InfoNCE, temp. τ)
sampling	adaptive log : $\alpha = 0.5$, $g_{\min} = \lfloor K/N \rfloor$, $g_{\max} = 64$
loss weight λ_g	tuned per configuration on validation (see Section 6.3)
relational order N	$\{3, 4, 8, 16, 17\}$ at $K=128$ ($N=2$ omitted, cf. Lemma 1; $N=3$ added for RKD-A)

for producing the results, not the results themselves. No headline outcome is claimed until the full protocol is run.]

Implementation guide: protocol invariants

Protocol invariants — do not violate. These make the comparison fair.

- I1 Per-configuration λ_g tuning** on *validation* (never global, never on test). The original failure was a fixed $\lambda_g=1000$ calibrated for the tiny $N=2$ loss, swamping the triplet term $\sim 500:1$ at active orders.
- I2 Classic baselines at prescribed weights + full RKD recipe:** RKD-D = 25, RKD-A = 50 (1:2), μ -normalization, Huber potential, RKD sampling. Do not re-tune them.
- I3 Everything else shared:** teacher, splits, budget, triplet task loss, warm-up, optimizer, $K=128$. Only the relational term differs.
- I4 Selection metric = validation mAP@R** everywhere (teacher, student, order). Recall@K reported, never selected on.
- I5 Multi-seed finals** (≥ 3 seeds, mean $\pm$ sem). No single-seed point estimates in the headline comparison.
- I6 Warm-up ramps to λ_g** , not to 1.
- I7 Per-term logging always on:** log `graph_loss` and `triplet_loss` separately every run (a silent zero must be detectable).
- I8 No mixup/cutmix** (breaks teacher/student node correspondence).

7.1 Quantitative Results

[Left blank. Produce here: per-dataset tables of test mAP@R / R-Precision / Recall@K (mean $\pm$ sem over ≥ 3 seeds) for the five students on both teachers, plus a grouped mAP@R bar chart. Analyze against H1 using the seed noise floor and the consistency-of-sign rule above.]

7.2 The Relational Order N and Normalization

[Left blank. Produce here: the λ_g -viability plot (H0 gate); the normalization ablation table/bars (H2); and the per-order characterization — mAP@R, convergence, stability, and cost as functions

of $N \in \{3, 4, 8, 16, 17\}$ — with an explicit best-by-trade-off recommendation. $N = 2$ is excluded by Lemma 1.]

7.3 Descriptor and Objective Characterization

[Left blank. Produce here: the profile-vs-MDS per-scenario table with the stability and fidelity instrumentation (H3), and the regression-vs-contrastive λ_g -robustness overlay with the width quantification and negative-quality diagnostics (H4). Give scenario recommendations only where gaps clear the noise floor and are mechanistically explained.]

7.4 The $N = 3$ Comparison against RKD-A

[Left blank. Produce here: the matched-arity $N=3$ comparison (profile and MDS vs RKD-A) per dataset/teacher, and its interpretation for whether an angular descriptor is warranted.]

7.5 Qualitative Analysis

[Left blank. Produce here: top-k retrieval panels (successes and failures) for Graph-RKD vs the strongest classic baseline, with the error-modes-not-ranking caveat.]

8 Discussion

Our method is best understood against the evolution of knowledge distillation, whose progression concerns *what* is taken to constitute the teacher’s knowledge: output logits [4], intermediate feature maps [5], spatial attention [6], and then the *relations* among examples in Relational KD [7]. Rather than re-narrate that progression (Sections 1 and 2), we position Graph-RKD by what it *subsumes*: modeling a minibatch as order-invariant graphs generalizes relational transfer to arbitrary order, and the low orders recover known methods — $N = 1$ is individual (unary) distillation, and $N = 2$ is distance-wise RKD (Lemma 1) — so $N \geq 3$ is the genuinely new, higher-order regime. In this sense Graph-RKD is a strict generalization of RKD’s own order hierarchy, and the study is framed as a characterization of its design space (normalization, descriptor, objective) at the orders where it is a distinct method.

One methodological lesson is worth stating regardless of the empirical outcome: in a multi-term distillation loss, a term can zero out *silently*. Only per-term logging (`train/graph_loss`) revealed that, under per-graph normalization, the graph term vanishes identically at $N = 2$; we recommend such monitoring as standard practice, and it is the reason the normalization axis and Lemma 1 are central to this work rather than an afterthought.

[Discussion of empirical findings to be completed once the experiments of Section 7 are available. Whether Graph-RKD at $N \geq 3$ matches or exceeds the RKD-D/RKD-A baselines is left open until the fair, re-tuned, multi-seed comparison decides it; no direction is presupposed here.]

[Left blank. Add one short evidence-backed paragraph per hypothesis H1–H5 and project-question Q1, using the analysis rules of Section 7. Preserve the subsumption framing and the per-term-logging lesson above.]

Relation to the project questions (Q1–Q5). *[The five project questions and their answers are to be finalized with the results. In brief: Q1 (which teacher transfers best) depends on the new numbers; Q2 targets the teacher’s post-GAP L_2 -normalized pooled embedding (a 192-, 512-, and 768-dimensional space for student and the two teachers, bridged with no projection layer); Q3 is the 0.67M-parameter student (a 16–43 $\times$ reduction); Q4 pairs the triplet task loss with the relational graph loss — the metric-learning counterpart of the cross-entropy-plus-regression composition in the brief; and Q5 is the adaptation of RKD [7] that constitutes the method. We note that the project brief describes a classification setup, whereas we interpret the task in RKD’s native metric-learning setting; this reframes Q2–Q4 as above and is licensed by Q5’s invitation to adapt to [7].]*

9 Difficulties and Solutions

Degenerate normalization at $N = 2$. The most instructive technical difficulty was found through per-term logging. Per-graph normalization divides each distance matrix by the mean of its own off-diagonal entries (Algorithm 2), mirroring the classic RKD distance normalization — but a two-node graph has a single edge, so the matrix is divided by itself and every pair of examples maps to the constant matrix $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, with fixed descriptors (`profile` = (1, 1); `mds` eigenvalues = (0.5, 0)). Teacher and student descriptors coincide, and the graph loss and its gradient are identically zero. The failure is silent: training proceeds normally on the triplet term, and only the per-term log (`train/graph_loss` = 0 across all $N = 2$ runs) exposed it, later confirmed by the derivation of Lemma 1 and by direct execution of the loss code. Rather than a defect to patch after the fact, this motivated treating normalization as a first-class design axis (Section 4.6): under minibatch normalization the $N = 2$ descriptor provably recovers RKD-D (Lemma 1), so we omit $N = 2$ from Graph-RKD altogether and let the RKD-D baseline represent the pairwise case, evaluating Graph-RKD only at $N \geq 3$.

Combinatorial growth of higher-order relations. The number of ordered N -tuples in a minibatch grows super-exponentially with the relational order (Eq. 6), which makes a naive higher-order extension of RKD intractable. We addressed this by modeling each minibatch as order-invariant graphs, reducing the count to a binomial coefficient (Eq. 7), and by sampling a bounded number of graphs per step (Algorithm 1) so that the per-step cost is independent of the size of the graph space.

Comparing embedding spaces of different dimensionality. The teacher and student produce pooled representations of different widths (512 and 768 for the teachers, 192 for the student), so their geometries are not directly comparable. Our permutation-invariant graph descriptors have a dimensionality that depends only on the relational order N , not on the representation width, so teacher and student embeddings inhabit the same descriptor space and require no projection layer;

normalizing the distance matrix (under any of the schemes studied) further removes scale mismatch.

Preserving per-example correspondence. The graph loss must pair the student and teacher on identical node sets. This rules out augmentations that mix samples, such as mixup and cutmix, which would break the correspondence; we therefore exclude them from the distillation runs (Section 6.1). This choice also means the teacher’s embeddings cannot be precomputed and cached (each epoch’s augmentation changes them); within a step, however, the $K \times K$ teacher distance matrix is computed once and sliced for the sampled graphs.

Experiment organization and resources. Studying normalization, descriptor, objective, and order jointly yields a large configuration grid; managing, tuning (λ_g per configuration), and analyzing it under limited time and compute required staging the runs (establishing active-order viability on a targeted slice before expanding) and using targeted comparisons rather than the full cross-product.

10 Conclusion and Future Work

This work asked whether Relational Knowledge Distillation can be generalized beyond pairs and triplets, and how the resulting method compares to RKD at the orders where it is distinct. We proposed Graph-RKD, which models each minibatch as complete, undirected, weighted graphs over its examples; summarizes every graph with a permutation-invariant descriptor (sorted node profiles or the classical-MDS spectrum); keeps the loss tractable by sampling a bounded, adaptively sized set of graphs at each step; and studies three design axes — normalization, descriptor, and objective — comparing Graph-RKD at $N \geq 3$ against the RKD-D and RKD-A baselines under a shared, fair protocol. The formulation reduces the space of order- N relations from a falling factorial to a binomial coefficient and, because descriptor dimensionality depends only on N , requires no projection layer between teacher and student spaces. Analytically, we showed (Lemma 1) that the pairwise case $N = 2$ either degenerates or reduces exactly to RKD-D, so Graph-RKD recovers RKD’s low orders as special cases and is a strict higher-order generalization.

[The empirical conclusion — whether Graph-RKD at $N \geq 3$ matches or exceeds the RKD-D/RKD-A baselines, and which normalization, descriptor, and objective are preferable in which regime — is to be stated once the experiments of Section 7 are complete. We do not presuppose a positive or negative outcome.]

The principal direction for future work is a rule $N^*(K)$ predicting the quality-optimal relational order from the minibatch size, which would require sweeping K across datasets, teachers, descriptors, objectives, and normalization schemes — a study in its own right, for which the fixed- K per-order characterization here is the groundwork. A second direction, motivated by RKD-A’s angular relation, is to enrich the graphs with angular attributes (e.g. per-node angular profiles or a hybrid distance-plus-angle descriptor), bringing the information exploited by RKD-A into the graph formulation and subsuming both RKD-D and RKD-A as low-order special cases within a single descriptor.

References

- [1] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016.
- [3] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [4] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [5] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, and Y. Bengio, “FitNets: Hints for thin deep nets,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [6] S. Zagoruyko and N. Komodakis, “Paying more attention to attention: Improving the performance of convolutional neural networks via attention transfer,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [7] W. Park, D. Kim, Y. Lu, and M. Cho, “Relational knowledge distillation,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [8] J. Krause, M. Stark, J. Deng, and L. Fei-Fei, “3d object representations for fine-grained categorization,” in *IEEE International Conference on Computer Vision Workshops (ICCVW)*, 2013.
- [9] C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, “The Caltech-UCSD Birds-200-2011 dataset,” Tech. Rep. CNS-TR-2011-001, California Institute of Technology, 2011.
- [10] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.
- [11] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *International Conference on Machine Learning (ICML)*, 2020.
- [12] K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum contrast for unsupervised visual representation learning,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [13] Y. Tian, D. Krishnan, and P. Isola, “Contrastive representation distillation,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [14] M. Gutmann and A. Hyvärinen, “Noise-contrastive estimation: A new estimation principle for unnormalized statistical models,” in *International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2010.

- [15] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed representations of words and phrases and their compositionality,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.
- [16] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [17] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, “Graph convolutional neural networks for web-scale recommender systems,” in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2018.
- [18] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola, “Deep sets,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [19] W. S. Torgerson, “Multidimensional scaling: I. theory and method,” *Psychometrika*, vol. 17, no. 4, pp. 401–419, 1952.
- [20] T. F. Cox and M. A. A. Cox, *Multidimensional Scaling*. Chapman and Hall/CRC, 2nd ed., 2000.
- [21] K. Musgrave, S. Belongie, and S.-N. Lim, “A metric learning reality check,” in *European Conference on Computer Vision (ECCV)*, 2020.